

CSS

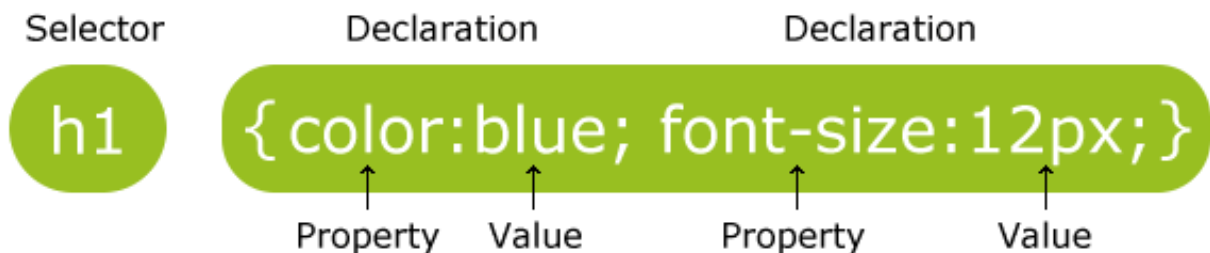
Introducción

CSS es un lenguaje de hoja de estilo que describe la presentación de un documento HTML. CSS describe como los elementos deben ser presentados en la pantalla, en papel o en algún otro medio.

- **CSS** es el acrónimo de **C**ascading **S**tyle **S**heet, hoja de estilos en cascada.
- CSS describe como los elementos HTML son presentados en la pantalla, papel o algún otro medio.
- CSS nos ahorra mucho trabajo debido a que permite controlar el estilo de muchas páginas simultáneamente.
- Las hojas de estilo externas son almacenadas en **archivos CSS**.
- La W3C propone separar el contenido, la funcionalidad y el estilo de una página web de manera que sean entidades independientes que interactúan simultáneamente. El contenido de una página se escribe en HTML5, el estilo se controla en CSS3 y la funcionalidad se codifica en JavaScript.

Sintáxis

El conjunto de reglas CSS consiste de un selector y un bloque de declaración:



El selector apunta al elemento HTML que se desea modificar el estilo. El bloque de declaración se deben separar las propiedades con punto y coma. Cada declaración incluye una propiedad CSS, nombre y valor separados por dos puntos. Una declaración CSS siempre termina con punto y coma, y los bloques de declaración están delimitados por llaves.

Selectores en CSS

Los selectores en CSS se utilizan para marcar elementos HTML de acuerdo a su nombre, id, clase, atributo entre otros.

Selección por elemento

La selección por elemento se basa en el nombre del elemento. Por ejemplo, se pueden seleccionar todos los elementos <p> y definir que deberán estar centrados y con texto color rojo.

```
p {  
  color: red;  
  text-align: center;  
}
```

De esta forma todos los elementos que se encuentren en el documento HTML tendrán asignado este estilo.

Selección por id

El selector por id utiliza este atributo de los elementos HTML para seleccionar un elemento específico. El id de un elemento debería ser único en una sola página, por lo tanto la selección por id debería ocurrir a un único elemento.

Para seleccionar un elemento desde CSS con un id específico, se debe escribir el caracter # seguido del id del elemento.

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

Nota: El nombre de un id no puede comenzar con un número.

Selección por clase

El selector por clase marca los elementos con un nombre específico de clase. El atributo clase se utiliza cuando se desean marcar dos o más elementos HTML para asignarles un estilo específico.

Para seleccionar elementos desde CSS con una clase específica, se debe escribir el caracter . seguido del nombre de la clase.

```
.center {  
  text-align: center;  
  color: red;  
}
```

A pesar de que la recomendación de la W3C indica que los id se utilicen para marcar un solo elemento HTML y las clases para marcar dos o más elementos HTML, no existe una prohibición que impida hacer lo contrario. Sin embargo esta recomendación tiene la intención de tener un contenido y estilo bien estructurado y ordenado.

Se pueden especificar solamente algunos elementos HTML específicos para ser afectados por el estilo asignado a la clase. Por ejemplo:

```
p.center {  
  text-align: center;  
  color: red;  
}
```

En este ejemplo, el estilo solo aplicará a los elementos <p> cuya clase sea center. Los elementos <p> que no tengan esa clase y los elementos que tengan clase center que no sean <p> no se verán afectados.

Los elementos HTML pueden tener más de una clase. En el siguiente ejemplo el elemento <p> recibirá el estilo definido por la clase center y large.

```
<p class="center large">  
  This paragraph refers to two classes.  
</p>
```

Nota: El nombre de una clase no puede comenzar con un número.

Agrupación de selectores

Si se tienen elementos con las mismas definiciones de estilo pueden ser agrupados para minimizar el código. Por ejemplo:

```
h1 {
  text-align: center;
  color: red;
}
h2 {
  text-align: center;
  color: red;
}
p {
  text-align: center;
  color: red;
}
```

Estos elementos pueden ser agrupados, separando por comas los selectores de los elementos.

```
h1, h2, p {
  text-align: center;
  color: red;
}
```

Comentarios en CSS

Los comentarios son utilizados para explicar el código, ayuda a entender el código en una edición futura. Los comentarios son ignorados por los navegadores. Los comentarios en CSS se inician con /* y se finalizan con */, pueden marcarse cualquier número de líneas.

```
p {
  color: red;
  /* This is a single-line comment */
  text-align: center;
}
/* This is
a multi-line
comment */
```

Modos de inserción de estilo

Existen tres formas de insertar estilo:

- Externa.
- Interna.
- En línea.

Hoja de Estilo Externa

Una hoja de estilos externa permite controlar la apariencia de una página Web completa con tan solo un archivo.

Cada página debe incluir una referencia a la hoja de estilos externa dentro de la etiqueta HTML `<link>`. Esta etiqueta debe incluirse dentro de la sección de configuración `<head>`.

```
<head>
  <link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

En este ejemplo, el atributo `href` hace referencia al archivo llamado `mystyle.css` que contiene el estilo externo.

Una hoja de estilos externa puede ser escrita en un editor de texto y no debe incluir etiquetas HTML. El archivo de hoja de estilos debe tener extensión `.css`.

No añada espacios entre el valor de la propiedad y la unidad correspondiente (por ejemplo `margin-left:20 px;`), la forma correcta es: `margin-left:20px;`.

Hoja de Estilo Interna

Se recomienda utilizar una hoja de estilo interna si se tiene un solo archivo HTML. El estilo interno se define dentro de la etiqueta `<style>` dentro del encabezado `<head>` de la página HTML.

```
<head>
  <style>
    body {
      background-color: linen;
    }

    h1 {
      color: maroon;
      margin-left: 40px;
    }
  </style>
</head>
```

Estilo En Línea

El estilo en línea se utiliza para establecer estilo a un único elemento HTML. Para ello basta con añadir el atributo `style` a la etiqueta HTML que se desea modificar el estilo. El atributo `style` puede contener cualquier propiedad CSS.

```
<h1 style="color:blue;margin-left:30px;">
  This is a heading.
</h1>
```

Utilizar este método para establecer el estilo ocasiona la pérdida de las ventajas de una hoja de estilos. Debería ser utilizado solo cuando no haya otra alternativa.

Orden en Cascada

En CSS el estilo se aplica en cascada, lo que significa que los estilos se aplican de acuerdo a su prioridad.

- Estilo en Línea
- Estilo en Hoja de Estilos Externa e Interna

- Estilo por defecto aplicado por el navegador.

La aplicación del estilo sobre el contenido en HTML funciona en cascada (tal como su nombre lo indica), esto significa que cuando un navegador recibe el código HTML se aplica el estilo por defecto del navegador. Posteriormente se buscan las hojas de estilo específicas para el documento y se aplica el estilo contenido en cada una de ellas. En primer lugar se busca la hoja de estilos externa, luego la hoja de estilos interna y finalmente el estilo en línea.

Prioridad de las Hojas de Estilo en HTML

1. En Línea
2. Interna
3. Externa
4. Predeterminada del Navegador

Orden en que se aplican las Hojas de Estilo

1. Predeterminada del Navegador
2. Externa
3. Interna
4. En Línea

CSS aplica el estilo en cascada, lo que significa que en primer lugar se aplica el estilo por defecto del navegador, posteriormente el estilo en la hoja externa, luego el estilo en la hoja interna y por último el estilo en línea. Conforme un estilo se aplica posterior a otro, se reemplazan los atributos que se encuentren repetidos de acuerdo a la prioridad que corresponde a cada tipo de estilo.

Estilo de tablas con CSS

El estilo de las tablas en CSS puede mejorarse notablemente.

Bordes de las Tablas

Para especificar bordes en la tabla se utiliza la propiedad `border`. El siguiente ejemplo establece una línea sólida de color negro de 1 pixel para los elementos `<table>`, `<th>` y `<td>`.

```
table, th, td {  
    border: 1px solid black;  
}
```

Observe que en este ejemplo se tienen bordes dobles, esto es debido a que cada uno de los elementos `<th>` y `<td>` tienen sus respectivos bordes.

Colapsar los bordes de las Tablas

La propiedad `border-collapse` establece si los bordes deben ser colapsados en una sola línea.

```
table {  
    border-collapse: collapse;  
}  
  
table, th, td {  
    border: 1px solid black;  
}
```

Si solo se desea un único borde alrededor de la tabla, solo se debería especificar la propiedad `border` para la etiqueta `<table>`.

```
table {  
    border: 1px solid black;  
}
```

Ancho y Alto de la Tabla

El ancho y alto de una tabla se definen con las propiedades `width` y `height` respectivamente.

En el siguiente ejemplo se establece un ancho de la tabla al 100% y el alto de los elementos `<th>` de 50 píxeles.

```
table {  
    width: 100%;  
}  
  
th {  
    height: 50px;  
}
```

Alineación Horizontal

La propiedad `text-align` establece la alineación horizontal (ya sea derecha, izquierda o centrada) del contenido de `<th>` y `<td>`. Por defecto, el contenido de los elementos `<th>` está centrado y el contenido de los elementos `<td>` está alineado a la izquierda.

```
th {  
    text-align: left;  
}
```

Alineación Vertical

La propiedad `vertical-align` establece la alineación vertical (ya sea superior, inferior o media) del contenido de `<th>` y `<td>`. Por defecto la alineación vertical del contenido de la tabla es inferior, tanto para los elementos `<th>` como para `<td>`.

```
td {  
    height: 50px;  
    vertical-align: bottom;  
}
```

Espaciado en la Tabla

Para controlar el espaciado interno del contenido de la tabla, se utiliza la propiedad `padding` en los elementos `<th>` y `<td>`.

```
th, td {  
    padding: 15px;  
    text-align: left;  
}
```

```
}
```

Divisores Horizontales

Para añadir solo divisores horizontales basta con establecer la propiedad `border-bottom` a los elementos `<th>` y `<td>`.

```
table {
  border-collapse: collapse;
  width: 100%;
}

th, td {
  padding: 8px;
  text-align: left;
  border-bottom: 1px solid #ddd;
}
```

Tabla con Resaltado al paso del Cursor

Para que una tabla reaccione al usuario cuando este pasa el puntero sobre alguno de sus elementos, basta con añadir el selector `:hover` a los elementos `<tr>`.

```
tr:hover {
  background-color: #f5f5f5
}
```

Tablas a Rayas

Para tablas que estén rayadas en sus renglones, se puede utilizar el selector `nth-child()` y añadir un color de fondo para todos los renglones pares o noes.

```
tr:nth-child(even) {
  background-color: #f2f2f2
}
```

Colores en las Tablas

El ejemplo muestra una tabla en la cual los elementos `<th>` tienen un color de fondo específico.

```
th {
  background-color: #4CAF50;
  color: white;
}
```

Tablas Responsivas

Una tabla responsiva despliega una barra de desplazamiento horizontal si la pantalla es muy pequeña para desplegar el contenido de la tabla.

Para conseguir esto basta con añadir un elemento contenedor (como puede ser `<div>`) alrededor del elemento `<table>` para hacerlo responsivo.

```

<!DOCTYPE html>
<html>
<head>
<style>
table {
    border-collapse: collapse;
    width: 100%;
}

th, td {
    text-align: left;
    padding: 8px;
}

tr:nth-child(even){background-color: #f2f2f2}
</style>
</head>
<body>

<h2>Responsive Table</h2>
<p>A responsive table will display a horizontal scroll bar if the screen is too
small to display the full content. Resize the browser window to see the effect:</p>
<p>To create a responsive table, add a container element (like div) with
<strong>overflow-x:auto</strong> around the table element:</p>

<div style="overflow-x:auto;">
    <table>
        <tr>
            <th>First Name</th>
            <th>Last Name</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
            <th>Points</th>
        </tr>
        <tr>
            <td>Jill</td>
            <td>Smith</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
            <td>50</td>
        </tr>
    </table>
</div>

```

```

<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
  <td>94</td>
</tr>
<tr>
  <td>Adam</td>
  <td>Johnson</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
  <td>67</td>
</tr>
</table>
</div>

</body>
</html>

```

Texto en CSS

Color del Texto

La propiedad color se utiliza para establecer el color del texto.

En CSS, el color puede ser especificado por:

- Valor Hexadecimal: #ff0000
- Valor en RGB: rgb(255,0,0)
- Nombre del color: red

El color por defecto del texto en una página está definido por el selector body.

```

body {
  color: blue;
}
h1 {
  color: green;
}

```

Nota: La convención de CSS sugerida por la W3C dice que si se define la propiedad `color`, también debe definirse la propiedad `background-color`.

Alineación de Texto

La propiedad `text-align` se utiliza para establecer la alineación horizontal de un texto. Un texto puede ser alineado a la derecha, izquierda, centrado o justificado.

```
h1 {
    text-align: center;
}

h2 {
    text-align: left;
}

h3 {
    text-align: right;
}
div {
    text-align: justify;
}
```

Decoración de texto

La propiedad `text-decoration` se utiliza para establecer o quitar decoración al texto. El valor `text-decoration: none`; se utiliza comúnmente para quitar el subrayado en el texto de los enlaces.

```
a {
    text-decoration: none;
}
```

Los demás valores posibles que puede tener la propiedad `text-decoration` son los siguientes:

```
h1 {
    text-decoration: overline;
}

h2 {
    text-decoration: line-through;
}

h3 {
    text-decoration: underline;
}
```

Nota: No se recomienda utilizar subrayado en texto que no sea un enlace, esto puede confundir a los lectores del sitio.

Estilo de Enlaces

El estilo de los enlaces pueden ser modificados por cualquier propiedad de CSS. Adicionalmente, el estilo de los enlaces puede ser modificado de acuerdo al estado en el que se encuentran. Los estados son:

- `a:link`. Enlace no visitado
- `a:visited`. Enlace ya visitado
- `a:hover`. Cuando el usuario pasa el ratón sobre el enlace.
- `a:active`. El momento cuando el usuario presiona el enlace.

```
/* unvisited link */
a:link {
    color: red;
}

/* visited link */
a:visited {
    color: green;
}

/* mouse over link */
a:hover {
    color: hotpink;
}

/* selected link */
a:active {
    color: blue;
}
```

Cuando se modifique el estilo de muchos enlaces, hay un par de reglas que deben tomarse:

- `a:hover` debe definirse después de `a:link` y `a:visited`
- `a:active` debe definirse después de `a:hover`

Estilo Avanzado de Enlaces

El siguiente ejemplo demuestra un estilo más avanzado donde se combinan varias propiedades CSS para desplegar los enlaces como botones que reaccionan al usuario.

```
<!DOCTYPE html>
<html>
<head>
<style>
    /* Estilo en común para todos los enlaces */
    a {
        color: white;
        padding: 14px 25px;
        text-decoration: none;
        text-align: center;
        border: 2px solid white;
        display: inline-block;
    }
    /* Estilo para cada enlace */
    a#link1 {
        background-color: rgb(154, 55, 55);
    }
    a#link2 {
```

```

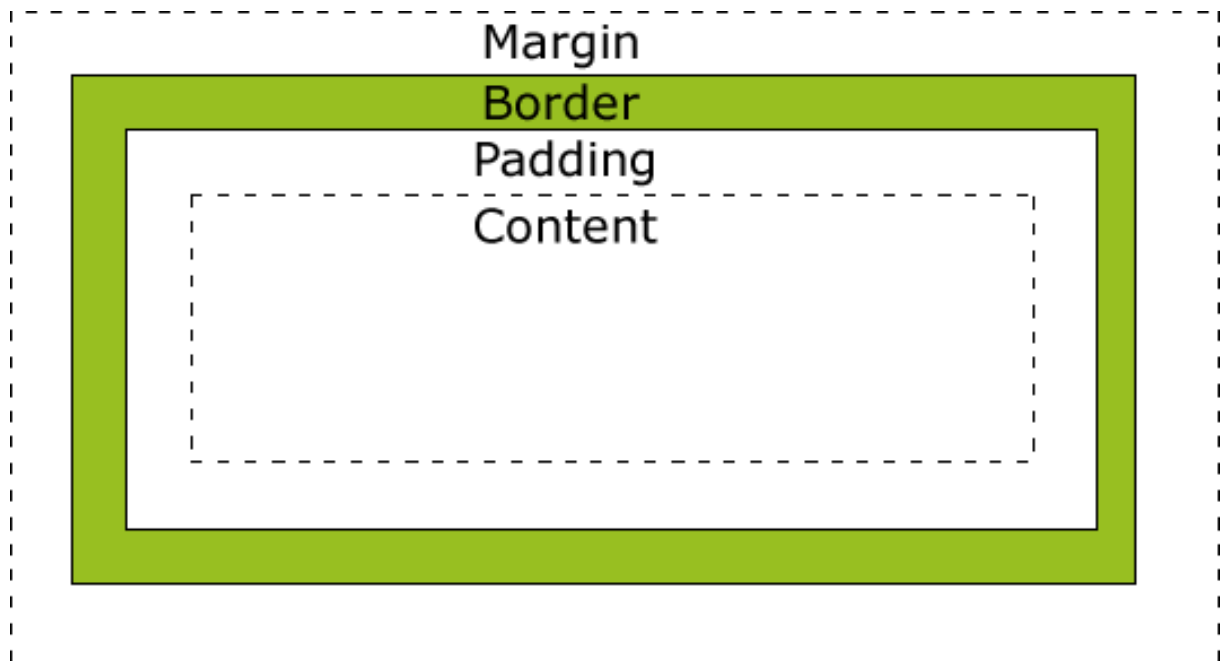
        background-color: green;
    }
    a#link3 {
        background-color: blue;
    }
    /* Comportamiento cuando el cursor está sobre algún enlace */
    a#link1:hover, a#link2:hover, a#link3:hover {
        color: black;
        background-color: white;
        border: 2px solid red;
    }
</style>
</head>
<body>
    <a id="link1" href="javascript:void(0)">Enlace 1</a>
    <a id="link2" href="javascript:void(0)">Enlace 2</a>
    <a id="link3" href="javascript:void(0)">Enlace 3</a>
</body>
</html>

```

Modelo de Cajas

Todos los elementos en HTML pueden ser considerados como cajas. En CSS en término “modelo de caja” se utiliza cuando se trata acerca del diseño y estilo.

El modelo de cajas consiste de márgenes, bordes, espaciado y contenido.



- **Margin:** Establece un área alrededor de un borde. El margen no tiene un color de fondo, es transparente.
- **Border:** El borde que se encuentra entre el espaciado y el contenido. El borde es heredado de la propiedad color de la caja.
- **Padding:** Establece un área alrededor del contenido. El espaciado es afectado por el color de fondo de la caja.
- **Content:** El contenido de la caja, el lugar donde está el texto o imágenes.

Ejemplo 1. Modelo de cajas de CSS

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Título</title>
  <style>
    p.x {
      width: 220px;
      padding: 10px;
      border: 5px solid gray;
      margin: 0px;
    }
  </style>
</head>
<body>
  <p>Ejemplo de Box Model</p>
  <p class="x">Este texto tiene como ancho de 220 pixeles, espaciado de 10 pixeles,
  bordes de 5 pixeles grises sólidos y margen de 0 pixeles.</p>
</body>
</html>
```

Border

La propiedad **border-style** especifica que tipo de borde se desplegará.

- **Border Width.** Define el ancho del borde. Por defecto se utiliza pixeles pero se puede utilizar valores: thin, medium o thick.

```
p.one {
  border-style: solid;
  border-width: 5px;
}

p.two {
  border-style: solid;
  border-width: medium;
}
```

- **Border Color.** Define el color del borde. El color puede ser establecido como:
 - **nombre.** Especificar el color por su nombre. e.g. "red"
 - **RGB.** Especificar el valor RGB. e.g. "rgb(255,0,0)"
 - **Hex.** Especificar el valor hexadecimal. e.g. "#ff0000"

```
p.one {
  border-style:solid;
  border-color:red;
}

p.two {
  border-style:solid;
```

```
border-color:#98bf21;
}
```

En CSS es posible especificar diferentes bordes para todos los lados:

```
p {
border-top-style:dotted;
border-right-style:solid;
border-bottom-style:dotted;
border-left-style:solid;
}
```

Con la finalidad de reducir la cantidad de código, es posible especificar todos los bordes en uno solo. Sin embargo, es posible especificar el borde por separado por si fuera necesario. Por lo tanto, esta propiedad puede tener de uno a cuatro valores:

- border-style: dotted solid double dashed;
 - borde superior = punteado
 - borde derecho = sólido
 - borde inferior = doble
 - borde izquierdo = sombreado
- border-style: dotted solid double;
 - borde superior = punteado
 - borde derecho e izquierdo = sólido
 - borde inferior = doble
- border-style: dotted solid;
 - borde superior e inferior = punteado
 - borde derecho e izquierdo= sólido
- border-style: dotted;
 - borde superior, inferior, derecho e izquierdo = punteado

Margin

El margen es el área alrededor de un elemento (fuera del borde). El margen no tiene color de fondo y es transparente.

Posibles valores:

Valor	Descripción
auto	El navegador calcula el margen
length	Especifica el margen en px, pt, cm, etc. El valor por defecto es 0px.
%	Especifica un margen en porcentaje del ancho del elemento contenido.
inherit	Especifica que el margen debe ser heredado del elemento padre.

En CSS es posible especificar diferente margen para cada lado de la caja. Por ejemplo:

```
clase/id/elemento {  
  margin-top:100px;  
  margin-bottom:100px;  
  margin-right:50px;  
  margin-left:50px;  
}
```

Con la finalidad de reducir la cantidad de código, es posible especificar todos los márgenes en una sola línea. Sin embargo, es posible especificar el margen por separado por si fuera necesario. Por lo tanto, esta propiedad puede tener de uno a cuatro valores:

- margin: 25px 50px 75px 100px;
 - margen superior = 25px
 - margen derecho = 50px
 - margen inferior = 75px
 - margen izquierdo = 100px
- margin: 25px 50px 75px;
 - margen superior = 25px
 - margen derecho e izquierdo = 50px
 - margen inferior = 75px
- margin: 25px 50px;
 - margen superior e inferior = 25px
 - margen derecho e izquierdo = 50px
- margin: 25px;
 - margen superior, inferior, derecho e izquierdo = 25px

Padding

En CSS es posible especificar diferente espaciado para cada lado de la caja. Por ejemplo:

```
padding-top:25px;  
padding-bottom:25px;  
padding-right:50px;  
padding-left:50px;
```

Con la finalidad de reducir la cantidad de código, es posible especificar todos los espaciados en una sola línea. Sin embargo, es posible especificar el espaciado por separado por si fuera necesario. Por lo tanto, esta propiedad puede tener de uno a cuatro valores:

- padding: 25px 50px 75px 100px;
 - espaciado superior = 25px
 - espaciado derecho = 50px
 - espaciado inferior = 75px

- espaciado izquierdo = 100px
- padding: 25px 50px 75px;
 - espaciado superior = 25px
 - espaciado derecho e izquierdo = 50px
 - espaciado inferior = 75px
- padding: 25px 50px;
 - espaciado superior e inferior = 25px
 - espaciado derecho e izquierdo = 50px
- padding: 25px;
 - espaciado superior, inferior, derecho e izquierdo = 25px

Desplegado en CSS

La propiedad `display` en CSS es la más importante para controlar el diseño. La propiedad `display` especifica si un elemento es desplegado y la forma como se despliega.

Cada elemento en HTML tiene un valor por defecto de `display` dependiendo del elemento que sea. El valor por defecto para la mayoría de los elementos es `block` o `inline`.

Click to show panel

This panel contains a `<div>` element, which is hidden by default (`display: none`).

It is styled with CSS, and we use JavaScript to show it (change it to (`display: block`)).

Elementos en Bloque

Un elemento en bloque siempre comienza en una nueva línea y toma todo el ancho disponible (se estira hacia la derecha e izquierda tanto como sea posible).

Ejemplos de elementos en bloque:

- `<div>`
- `<h1>` hasta `<h6>`
- `<p>`
- `<form>`
- `<header>`
- `<footer>`
- `<section>`

Elementos en Línea

Un elemento en línea no inicia en una nueva línea y solo toma el espacio necesario.

Ejemplos de elementos en línea:

- `<span>`
- `<a>`
- `<img>`

Display: none

El valor `display: none` se utiliza frecuentemente con JavaScript para ocultar y mostrar elementos sin borrarlos ni volverlos a crear. El elemento `<script>` utiliza `display: none` por defecto.

Es posible cambiar el valor del atributo `display` de los elementos para que estos se comporten de forma diferente a como lo harían por defecto. Cambiar el valor de en línea a bloque puede ser útil para hacer que una página luzca de una forma específica, sin violar los estándares de la web.

Un ejemplo típico de esto es en los elementos `<li>`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo Lista</title>
  <style>
    li.clase {
      display: inline;
    }
  </style>
</head>
<body>
  <h1>Lista en bloque</h1>
  <ul>
    <li>Elemento 1</li>
    <li>Elemento 2</li>
    <li>Elemento 3</li>
  </ul>

  <h1>Lista en Línea</h1>
  <ul>
    <li class="clase">Elemento 1</li>
    <li class="clase">Elemento 2</li>
    <li class="clase">Elemento 3</li>
  </ul>
</body>
</html>
```

Esconder un elemento: ¿display:none o visibility:hidden?

Para esconder un elemento se debe establecer a la propiedad `display: none`; el elemento se esconderá y la página se desplegará como si el elemento no estuviese allí.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1.hidden {
      display: none;
    }
  </style>
</head>
<body>
  <h1>This is a visible heading</h1>
  <h1 class="hidden">This is a hidden heading</h1>
```

```

    <p>Notice that the h1 element with display: none; does not take up any space.</p>
</body>
</html>

```

Por otro lado, `visibility: hidden;` también ocultará el elemento, sin embargo el elemento aún toma el espacio en el esquema de la página.

```

<!DOCTYPE html>
<html>
<head>
<style>
    h1.hidden {
        visibility: hidden;
    }
</style>
</head>
<body>
    <h1>This is a visible heading</h1>
    <h1 class="hidden">This is a hidden heading</h1>
    <p>Notice that the hidden heading still takes up space.</p>
</body>
</html>

```

Propiedad	Descripción
display	Especifica como sera desplegado un elemento.
visibility	Especifica si un elemento será desplegado o no.

Ejemplo 2. display: none Vs. visibility: none

```

<!DOCTYPE html>
<html>
<head>
<style>
    .imgbox {
        float: left;
        text-align: center;
        width: 120px;
        height: 145px;
        border: 1px solid gray;
        margin: 4px;
        padding: 0;
    }
    .thumbnail {
        width: 110px;
        height: 90px;
        margin: 3px;
    }
    .box {
        width: 110px;
        padding: 0;
    }
</style>
</head>

```

```

<body>
  <div style="text-align:center">
    <div style="width:394px;height:160px;margin-left:auto;
      margin-right:auto;text-align:left;border:1px solid gray;">
      <div class="imgbox" id="imgbox1">Box 1<br>
        
        <input class="box" type="button" onclick="removeElement()" value="Remove">
      </div>
      <div class="imgbox" id="imgbox2">Box 2<br>
        
        <input class="box" type="button" onclick="changeVisibility()"
value="Hide">
      </div>
      <div class="imgbox">Box 3<br>
        
        <input class="box" type="button" onclick="resetElement()" value="Reset
All">
      </div>
    </div>
  </div>

  <script>
    function removeElement() {
      document.getElementById("imgbox1").style.display = "none";
    }
    function changeVisibility() {
      document.getElementById("imgbox2").style.visibility = "hidden";
    }
    function resetElement() {
      document.getElementById("imgbox1").style.display = "block";
      document.getElementById("imgbox2").style.visibility = "visible";
    }
  </script>
</body>
</html>

```

Diseño CSS

Como se ha mencionado, un elemento de bloque siempre ocupará todo el espacio disponible, se estirará a la izquierda y derecha tanto como sea posible.

Estableciendo un valor para width en un elemento de bloque previene que este se extienda fuera de los límites del contenedor. Por tanto, se puede configurar los márgenes en auto para centrar el elemento dentro de su contenedor. El elemento tomará el espacio especificado en width y el espacio restante se divide en dos partes iguales para los márgenes derecho e izquierdo.

Este <div> tiene un width de 50 píxeles y el margen en auto.

Nota: El problema con este <div> ocurre cuando la ventana del navegador es más pequeña que el elemento width. El navegador entonces agregará una barra de desplazamiento horizontal.

Este <div> tiene un max-width de 500 píxeles y el margen en auto.

Este segundo <div> tiene el atributo max-width: 500px; que permite al navegador manejar este elemento para pantallas pequeñas.

Ejemplo 3. Diferencia entre width y max-width.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div.ex1 {
      width:500px;
      margin: auto;
      border: 3px solid #2194ad;
    }

    div.ex2 {
      max-width:500px;
      margin: auto;
      border: 3px solid #2194ad;
    }
  </style>
</head>
<body>
  <div class="ex1">Este div tiene width: 500px;</div>
  <br>
  <div class="ex2">Este div tiene max-width: 500px;</div>
  <p>
    <strong>Tip:</strong> Cambie el tamaño de la ventana del
    navegador a menos de 500 pixeles para ver la diferencia
    entre ambos elementos.
  </p>
</body>
</html>
```

Propiedad position

La propiedad position especifica el tipo de posicionamiento utilizado por un elemento. Existen cuatro diferentes valores para el atributo position:

- static
- relative
- fixed
- absolute

Los elementos pueden ser posicionados utilizando las propiedades top, bottom, left y right. Sin embargo, estas propiedades no funcionarán a menos que la propiedad position sea establecida previamente. Adicionalmente, funcionan de forma diferente dependiendo del valor position.

position: static;

Los elementos en HTML tienen el valor de position: static por defecto. Los elementos con este valor no son afectados por las propiedades top, bottom, left y right.

Un elemento con la propiedad position: static; no es posicionado de ninguna forma en particular, se posiciona de acuerdo al flujo normal de la página.

Este elemento <div> tiene position: static;

Ejemplo 4. position: static;

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div.static {
      position: static;
      border: 3px solid #2194ad;
    }
  </style>
</head>
<body>
  <h2>position: static;</h2>
  <p>
    Un elemento con la propiedad position: static; no es posicionado de ninguna
    forma en particular, se posiciona de acuerdo al flujo normal de la página.
  </p>
  <div class="static">
    Este elemento div tiene position: static;
  </div>
</body>
</html>
```

position: relative;

Un elemento con position: relative; es posicionado relativo a su posición normal. Estableciendo las propiedades top, bottom, left y right de un elemento posicionado relativamente ocasionará que sea ajustado alejado de su posición normal. Otro contenido no será ajustado para ocupar el espacio que el elemento haya dejado a su izquierda.

Este elemento <div> tiene position: relative;

Ejemplo 5. position: relative;

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div.relative {
      position: relative;
      left: 30px;
      border: 3px solid #2194ad;
    }
  </style>
</head>
<body>
  <h2>position: relative;</h2>
  <p>
    Un elemento con position: relative; se posiciona de forma relativa a su
    posición normal:
  </p>
```

```

<div class="relative">
  Este elemento div tiene position: relative;
</div>
</body>
</html>

```

position: fixed;

Un elemento con position: fixed; se posiciona relativamente a la ventana, lo que significa que se mantendrá en la misma posición aún y si la página se desplaza. Las propiedades top, bottom, left y right se utilizan para posicionar el elemento.

Un elemento fijo no deja espacio en la página donde normalmente estaría ubicado.

Ejemplo 6. position: fixed;

```

<!DOCTYPE html>
<html>
<head>
  <style>
    div.fixed {
      position: fixed;
      bottom: 0;
      right: 0;
      width: 300px;
      border: 3px solid #73AD21;
    }
  </style>
</head>
<body>
  <h2>position: fixed;</h2>
  <p>
    Un elemento con position: fixed; se posiciona relativamente a la ventana, lo
    que significa
    que se mantendrá en la misma posición aún y si la página se desplaza.
  </p>
  <div class="fixed">
    Este div tiene position: fixed;
  </div>
</body>
</html>

```

position: absolute;

Un elemento con position: absolute; se posiciona relativamente al punto más cercano a su antecesor, en lugar de posicionarlo relativamente a la ventana.

Ejemplo 6. position: absolute;

```

<!DOCTYPE html>
<html>
<head>
  <style>
    div.relative {
      position: relative;
      width: 400px;
      height: 200px;
      border: 3px solid #73AD21;
    }
  </style>
</head>
<body>
  <div class="relative">
    <div class="absolute">
      <p>position: absolute;</p>
    </div>
  </div>
</body>
</html>

```

```

}

div.absolute {
  position: absolute;
  top: 80px;
  right: 0;
  width: 200px;
  height: 100px;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>
  <h2>position: absolute;</h2>
  <p>
    An element with position: absolute; is positioned relative to the nearest
    positioned ancestor
    (instead of positioned relative to the viewport, like fixed):
  </p>
  <div class="relative">This div element has position: relative;
  <div class="absolute">This div element has position: absolute;</div>
</div>
</body>
</html>

```